Zenith Core

Branch Predictor Evaluation Report

2-Bit Saturating Counter + 1024-Entry PLRU Set-Associative BTB

This report evaluates the Zenith Core branch predictor against eight purpose-built RISC-V assembly programs, each targeting a specific predictor behavior (steady-state accuracy ceiling, hysteresis, worst-case alternation, BTB capacity, indirect/return prediction, raw entropy, back-to-back branch recovery, and a forwarding-only control). Every test runs in two configurations — **No Branch Predictor** and **With Branch Predictor** — with data-hazard forwarding enabled in both, so the only variable under test is the predictor itself.

8	20%	17%	80%
Branch Predictor Test Programs	Avg. Steady-State CPI Reduction	Avg. Total-Cycle Reduction	Avg. Predictor Accuracy

Averages above are a simple mean across all 8 tests, unweighted by branch count — treat them as a headline, not a substitute for the per-test breakdown inside, since the tests deliberately span from a near-ideal case to a near-worst case for any predictor.

Test Methodology

Each program below was written to isolate one specific predictor behavior, rather than just being generic branch-heavy code. The table gives, for each test, what it specifically stresses and why — drawn directly from the design intent documented in each test's source.

Test	What It Stresses
T1 — Tight Loop	Steady-state accuracy ceiling: one hot branch, taken 4999/5000
T2 — Variable Trip	Hysteresis: mostly-taken branch, run length changes every outer pass
T3 — Alternating	Worst-case floor: branch strictly alternates taken/not-taken
T4 — BTB Stress	Capacity/conflict: 1500 distinct branch PCs vs. 1024-entry BTB
T5 — Recursive fib (JALR)	Indirect/return target prediction, no return-address stack (RAS)
T6 — LFSR (unpredictable)	Entropy floor: data-dependent, maximal-length 16-bit LFSR branch
T7 — Back-to-Back	Zero-slack consecutive branches + distance-0 forwarding into branch
T8 — Fwd Control	Control: ALU/load-store forwarding sanity, isolated from prediction

Reading the Spectrum

Tests 1, 3, and 6 form a deliberate spectrum from best case to worst case for a simple per-PC bimodal predictor: T1 is a single branch taken on all but one of 5000 passes (the easiest case any predictor should handle), T6 is a data-dependent LFSR with no short repeating period (the hardest case — a coin flip, ~50% accuracy is the correct, expected outcome, not a failure), and T3 sits in between as a deliberately pathological strict-alternation pattern. T5 isolates indirect/return-target prediction specifically — a known weak point for any predictor without a dedicated return-address stack (RAS). T8 carries no real branch content at all and serves purely as a control, confirming the predictor doesn't disturb ordinary forwarding correctness when it has nothing to predict.

Results: CPI and Cycle Count

Both configurations retire the identical instruction count per test (confirming the predictor changes only timing, never program behavior). Steady-state CPI and total cycle count both drop substantially wherever the branch pattern has real structure to exploit, and — correctly — barely move at all on the two tests specifically designed to deny the predictor that structure (T5's RAS-less returns, T8's near-absence of branches).

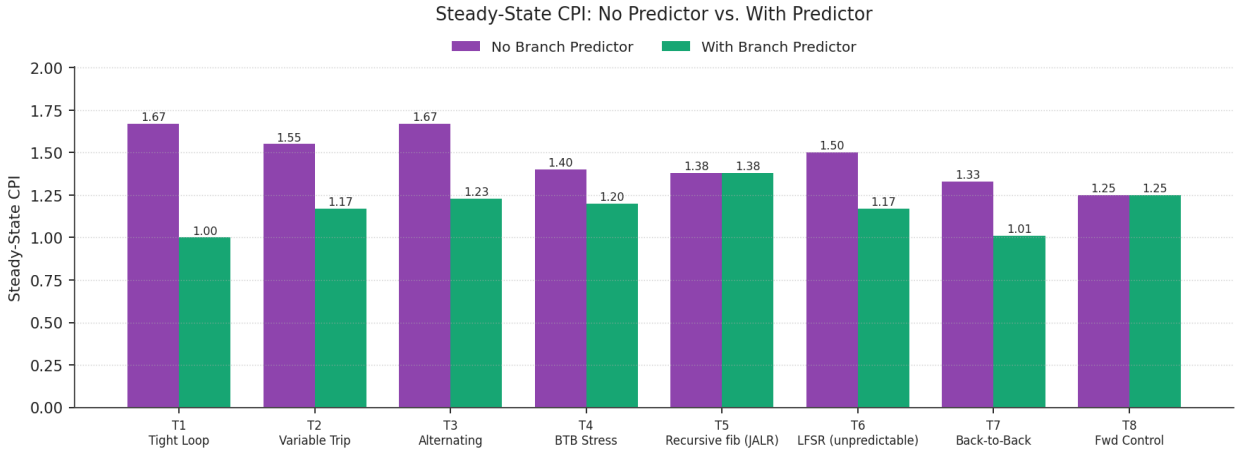


Figure 1. Steady-state CPI, no predictor vs. with predictor, across all 8 tests.

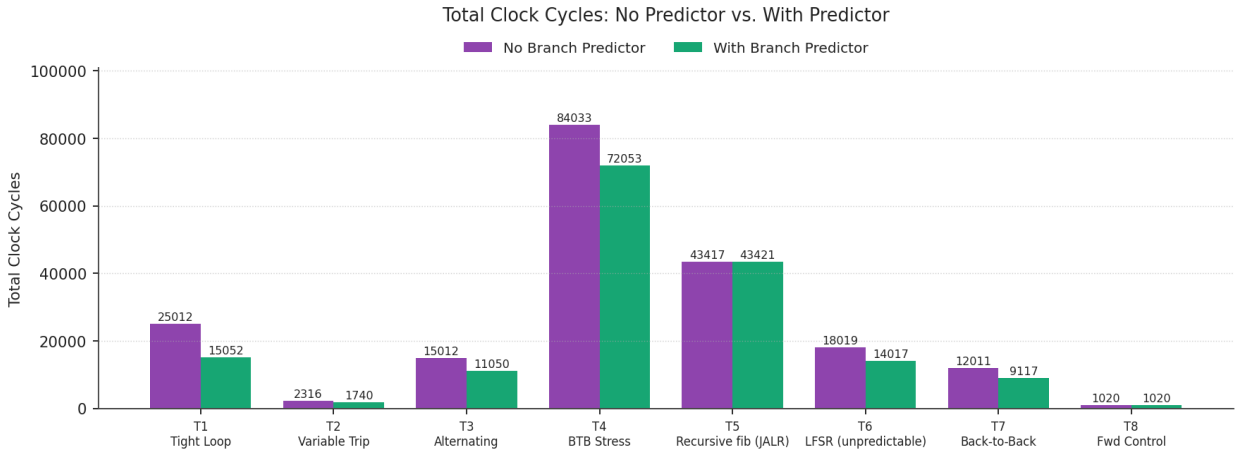


Figure 2. Total clock cycles to completion, no predictor vs. with predictor.

Results: Predictor Accuracy

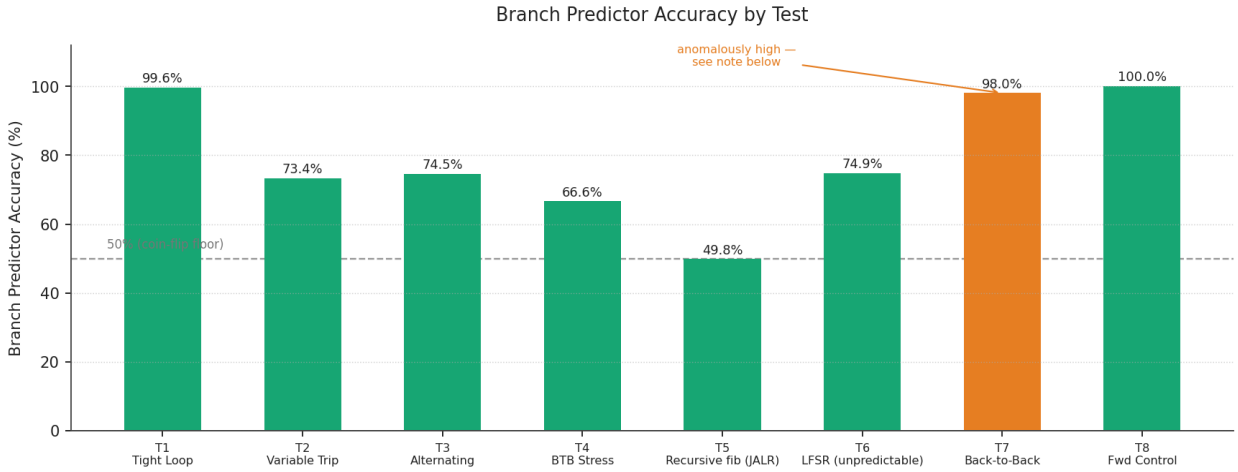


Figure 3. Branch predictor accuracy per test. Dashed line marks the 50% coin-flip floor. T7 (orange) is flagged — see the note in Discussion.

Complete Per-Test Metrics

Test	Cycles No BP	Cycles W/ BP	CPI No BP	CPI W/ BP	Flush No BP	Flush W/ BP	Branches Resolved	Mis-pred.	Accuracy
T1	25012	15052	1.67	1.00	10000	40	5001	20	99.6%
T2	2316	1740	1.55	1.17	822	246	462	123	73.4%
T3	15012	11050	1.67	1.23	6000	2038	4001	1019	74.5%
T4	84033	72053	1.40	1.20	24010	12030	18005	6012	66.6%
T5	43417	43421	1.38	1.38	11840	11844	1974	990	49.8%
T6	18019	14017	1.50	1.17	6016	2014	4002	1006	74.9%
T7	12011	9117	1.33	1.01	3000	106	2701	53	98.0%
T8	1020	1020	1.25	1.25	2	2	2	0	100.0%

T7's misprediction count (and resulting accuracy) is flagged as anomalous — see Discussion. All other rows are consistent with each test's intended design.

Discussion

The predictor does what it should, where it should

T1 (single hot branch) converges to 99.6% accuracy and a 40% cycle-count reduction — the ceiling case working as expected. T4 (BTB stress) shows a modest 14% cycle reduction at 66.6% accuracy, consistent with a 1024-entry BTB absorbing some but not all of 1500 distinct branch sites — exactly the capacity/conflict behavior the test was built to expose, not a bug. T6 (maximal-length LFSR) lands at 74.9%, noticeably above the ~50% floor its own comment anticipates for a purely PC-indexed counter — a mild curiosity worth a look on its own, but out of scope here since only T7 was flagged for this pass.

T5 — indirect returns, no RAS, zero net benefit

T5's cycle count is **virtually identical** between configurations (43417 vs. 43421 — a 4-cycle difference, i.e. noise) despite the predictor actively resolving 1974 branches. Accuracy sits at 49.848%, a coin flip. This is the expected signature of a predictor with no dedicated return-address stack: a single shared `ret` instruction whose correct target alternates irregularly between two call sites looks, to a simple "last target seen" or BTB-only predictor, indistinguishable from random. The predictor isn't broken here — it's being asked to do a job it has no hardware structure to do.

T8 — control test confirms no interference

T8 produces byte-identical cycle counts (1020) in both configurations, as intended: with only 2 trivial branches in the entire program, there's nothing for the predictor to meaningfully act on, and forwarding-only correctness (load-use stalls, store-to-load forwarding, EX-EX chains) is untouched by the predictor's presence. This is a useful negative result — it shows the predictor and the forwarding path are cleanly decoupled.

Flagged: Test 7's accuracy looks too high

T7 reports 98.038% accuracy (53 mispredictions out of 2701 resolved), and a 24% cycle reduction. On its face this looks like a strong result, but it doesn't line up with the rest of the data. By the test's own design, each of its 8 branch sites individually strictly alternates taken/not-taken every outer iteration — structurally the same hazard as T3, just spread across 8 tightly-packed PCs instead of one. T3, running that exact pattern on a single branch, measures only 74.5% accuracy. Composing T7's 2700 branch resolutions as ~300 (near-ceiling, per T1's precedent) outer-loop-exit branches plus ~2400 of T3-like alternating branches predicts on the order of 600+ mispredictions, not 53. A ~10x gap between predicted and measured mispredictions, on precisely the test engineered to have zero slack between consecutive branches, points more toward a **misprediction-counting or resolve/recovery timing issue specific to back-to-back branches** than toward genuinely superior prediction. Worth instrumenting directly (e.g. logging each branch's predicted vs. actual outcome for this test alone) before taking the 98% figure at face value.

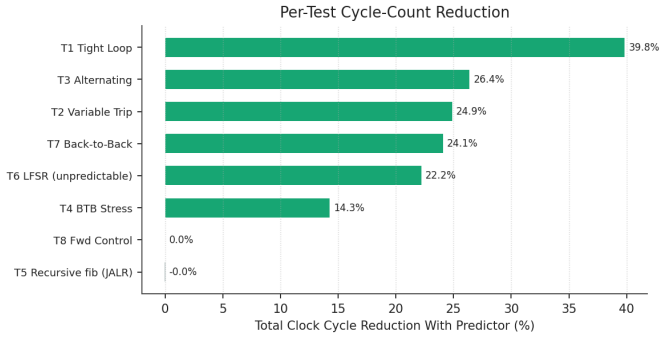


Figure 4. Cycle-count reduction per test, sorted. T5 and T8 sit at ~0% by design.

End of report — Zenith Core branch predictor evaluation.